# Zero to Hero questions for Arrays in C

June 16, 2026

## 0.1 Array Practice Questions (1D Arrays)

**Prerequisites:** `printf`, `scanf`, operators, conditions, loops, arrays

### 0.1.1 Level 1: Basic Input/Output

1. Input 5 integers and print them.
   - **Hint:** Use one loop for input and one loop for output.
2. Input `n` integers and print them in the same order.
   - **Hint:** Store in an array.
3. Input `n` integers and print them in reverse order.
   - **Hint:** Traverse from `n-1` to `0`.
4. Input `n` integers and print only the even numbers.
   - **Hint:** Use `% 2`.
5. Input `n` integers and print only the odd numbers.
   - **Hint:** Check remainder after division by 2.

---

### 0.1.2 Level 2: Sum and Counting

6. Find the sum of all elements.
   - **Hint:** Keep adding to a variable.
7. Find the average of all elements.
   - **Hint:** Average = Sum / Number of Elements.
8. Count positive numbers.
   - **Hint:** Check `> 0`.
9. Count negative numbers.
   - **Hint:** Check `< 0`.
10. Count zeros in the array.
    - **Hint:** Check `== 0`.

11. Count even and odd elements.
    - **Hint:** Maintain two counters.
12. Find the sum of even elements only.
    - **Hint:** Add only if even.
13. Find the sum of odd elements only.
    - **Hint:** Add only if odd.

---

#### 0.1.3 Level 3: Maximum and Minimum

14. Find the largest element.
    - **Hint:** Assume first element is largest.
15. Find the smallest element.
    - **Hint:** Assume first element is smallest.
16. Find both maximum and minimum in one traversal.
    - **Hint:** Update both variables.
17. Find the difference between largest and smallest elements.
    - **Hint:** Find max and min first.
18. Find the second largest element.
    - **Hint:** Maintain largest and second largest.
19. Find the second smallest element.
    - **Hint:** Maintain smallest and second smallest.

---

#### 0.1.4 Level 4: Searching

20. Search for a given number.
    - **Hint:** Compare each element.
21. Count occurrences of a given number.
    - **Hint:** Increment count whenever found.
22. Find the first occurrence of a number.
    - **Hint:** Stop when found.
23. Find the last occurrence of a number.
    - **Hint:** Keep updating position.
24. Check whether a number exists in the array.

- **Hint:** Use a flag variable.

---

### 0.1.5 Level 5: Array Modification

25. Copy one array into another.
    - **Hint:** Use a loop.
26. Multiply every element by 2.
    - **Hint:** Update each element.
27. Replace all negative numbers with 0.
    - **Hint:** Check condition before assignment.
28. Replace all even numbers with -1.
    - **Hint:** Check divisibility by 2.
29. Add 10 to every element.
    - **Hint:** Traverse entire array.

---

### 0.1.6 Level 6: Reversing

30. Reverse the array using another array.
    - **Hint:** Start from the end.
31. Reverse the array without another array.
    - **Hint:** Swap first and last elements.
32. Check whether an array is a palindrome.
    - **Hint:** Compare opposite ends.

---

### 0.1.7 Level 7: Sorting Basics

33. Sort array in ascending order.
    - **Hint:** Compare and swap.
34. Sort array in descending order.
    - **Hint:** Reverse comparison condition.
35. Find the largest element after sorting.
    - **Hint:** Observe last position.
36. Find the smallest element after sorting.
    - **Hint:** Observe first position.

---

### 0.1.8 Level 8: Frequency Problems

37. Count frequency of each element.
    - **Hint:** Nested loops.
38. Find the most frequent element.
    - **Hint:** Track highest frequency.
39. Find the least frequent element.
    - **Hint:** Track lowest frequency.
40. Find duplicate elements.
    - **Hint:** Compare every pair.
41. Count total duplicate elements.
    - **Hint:** Use nested loops.
42. Print all unique elements.
    - **Hint:** Frequency should be 1.

---

### 0.1.9 Level 9: Number Properties

43. Find the sum of digits of each element.
    - **Hint:** Use `% 10` and `/ 10`.
44. Count how many elements are prime.
    - **Hint:** Check divisibility.
45. Print all prime numbers in the array.
    - **Hint:** Prime-check function logic.
46. Count how many elements are perfect squares.
    - **Hint:** Check square root condition.
47. Count Armstrong numbers in the array.
    - **Hint:** Apply Armstrong logic to each element.
48. Count palindrome numbers in the array.
    - **Hint:** Reverse each number.

---

### 0.1.10 Level 10: Intermediate Challenges

49. Merge two arrays.
    - **Hint:** Copy elements one after another.
50. Find common elements between two arrays.
    - **Hint:** Compare each element of first array with second.
51. Find elements present in first array but not in second.
    - **Hint:** Search before printing.
52. Check whether two arrays are equal.
    - **Hint:** Compare corresponding elements.
53. Rotate array left by one position.
    - **Hint:** Store first element temporarily.
54. Rotate array right by one position.
    - **Hint:** Store last element temporarily.
55. Rotate array left by `k` positions.
    - **Hint:** Repeat left rotation.
56. Rotate array right by `k` positions.
    - **Hint:** Repeat right rotation.
57. Find all pairs whose sum equals a given number.
    - **Hint:** Nested loops.
58. Find all pairs whose product equals a given number.
    - **Hint:** Nested loops.
59. Find the missing number from 1 to n.
    - **Hint:** Compare expected sum and actual sum.
60. Find the element that appears only once.
    - **Hint:** Frequency count.

---

### 0.1.11 Mini Real-World Problems

61. Store marks of 10 students and find highest mark.
62. Store daily temperatures for a week and find average temperature.
63. Store monthly expenses and find total expense.
64. Store cricket scores and count centuries ( 100).
65. Store ages of people and count adults ( 18).
66. Store salaries and find highest salary.

67. Store rainfall data and find the rainiest day.
68. Store product prices and find the cheapest product.
69. Store electricity usage and calculate total consumption.
70. Store classroom attendance (1/0) and count present students.

These 70 questions can be solved using only **arrays + loops + conditions + operators + printf/scanf**, making them ideal before learning functions, pointers, and structures.

# 1 2D and 3D Array Practice Questions in C

**Prerequisites:** `printf`, `scanf`, operators, conditions, loops, 1D arrays

---

# 2 Level 1: Basic 2D Arrays (Matrices)

1. Input and display a 2×2 matrix.
   - **Hint:** Use nested loops.
2. Input and display a 3×3 matrix.
   - **Hint:** Row loop + column loop.
3. Find the sum of all elements in a matrix.
   - **Hint:** Add during traversal.
4. Find the average of all elements.
   - **Hint:** Divide total sum by rows × columns.
5. Find the largest element in a matrix.
   - **Hint:** Assume first element is maximum.
6. Find the smallest element in a matrix.
   - **Hint:** Assume first element is minimum.
7. Count positive elements.
   - **Hint:** Check `> 0`.
8. Count negative elements.
   - **Hint:** Check `< 0`.
9. Count zeros in the matrix.
   - **Hint:** Check `== 0`.
10. Count even and odd elements.
    - **Hint:** Use `% 2`.

---

## 3 Level 2: Row and Column Operations

11. Find the sum of each row.
    - **Hint:** Reset sum for every row.
12. Find the sum of each column.
    - **Hint:** Fix column, vary row.
13. Find the average of each row.
    - **Hint:** Row sum / number of columns.
14. Find the average of each column.
    - **Hint:** Column sum / number of rows.
15. Find the row with the maximum sum.
    - **Hint:** Compare row sums.
16. Find the column with the maximum sum.
    - **Hint:** Compare column sums.
17. Find the row with the minimum sum.
    - **Hint:** Track smallest row sum.
18. Find the column with the minimum sum.
    - **Hint:** Track smallest column sum.

---

## 4 Level 3: Diagonal Operations

19. Find the main diagonal sum.
    - **Hint:** Condition `i == j`.
20. Find the secondary diagonal sum.
    - **Hint:** Condition `i + j == n - 1`.
21. Print main diagonal elements.
    - **Hint:** Same row and column index.
22. Print secondary diagonal elements.
    - **Hint:** Sum of indices.
23. Find the difference between diagonal sums.
    - **Hint:** Compute both sums first.
24. Find the product of main diagonal elements.
    - **Hint:** Initialize product as 1.

---

## 5 Level 4: Matrix Transformations

25. Find the transpose of a matrix.
    - **Hint:** Swap row and column indices.
26. Print matrix in reverse row order.
    - **Hint:** Traverse rows backward.
27. Print matrix in reverse column order.
    - **Hint:** Traverse columns backward.
28. Mirror matrix horizontally.
    - **Hint:** Reverse columns.
29. Mirror matrix vertically.
    - **Hint:** Reverse rows.
30. Rotate matrix by 90° clockwise.
    - **Hint:** Transpose + reverse rows.
31. Rotate matrix by 90° anti-clockwise.
    - **Hint:** Transpose + reverse columns.

---

## 6 Level 5: Matrix Arithmetic

32. Add two matrices.
    - **Hint:** Add corresponding elements.
33. Subtract two matrices.
    - **Hint:** Subtract corresponding elements.
34. Multiply a matrix by a scalar.
    - **Hint:** Multiply every element.
35. Multiply two 2×2 matrices.
    - **Hint:** Row × Column.
36. Multiply two 3×3 matrices.
    - **Hint:** Use 3 nested loops.
37. Find element-wise multiplication.
    - **Hint:** Multiply corresponding elements.

38. Find element-wise division.
    - **Hint:** Divide corresponding elements.
39. Compute matrix square (A × A).
    - **Hint:** Matrix multiplication.
40. Compute matrix cube (A × A × A).
    - **Hint:** Multiply result again.

---

## 7 Level 6: Matrix Properties

41. Check whether matrix is square.
    - **Hint:** Rows == Columns.
42. Check whether matrix is symmetric.
    - **Hint:** Compare with transpose.
43. Check whether matrix is identity.
    - **Hint:** Diagonal 1, others 0.
44. Check whether matrix is diagonal.
    - **Hint:** Non-diagonal elements must be 0.
45. Check whether matrix is upper triangular.
    - **Hint:** Elements below diagonal are 0.
46. Check whether matrix is lower triangular.
    - **Hint:** Elements above diagonal are 0.
47. Check whether matrix is sparse.
    - **Hint:** Count zeros.
48. Check whether matrix is a zero matrix.
    - **Hint:** All elements must be 0.
49. Check whether matrix is equal to another matrix.
    - **Hint:** Compare corresponding elements.
50. Check whether matrix is skew-symmetric.
    - **Hint:** Compare with negative transpose.

---

## 8 Level 7: Searching in Matrices

51. Search for a number in a matrix.
    - **Hint:** Traverse all elements.
52. Count occurrences of a number.
    - **Hint:** Increment counter.
53. Find first occurrence position.
    - **Hint:** Store row and column.
54. Find last occurrence position.
    - **Hint:** Update whenever found.
55. Find all positions of a given element.
    - **Hint:** Print coordinates.

---

## 9 Level 8: Frequency Problems

56. Count frequency of each element.
    - **Hint:** Nested loops.
57. Find most frequent element.
    - **Hint:** Highest count.
58. Find least frequent element.
    - **Hint:** Lowest count.
59. Find duplicate elements.
    - **Hint:** Compare all pairs.
60. Print unique elements.
    - **Hint:** Frequency equals 1.

---

## 10 Level 9: Advanced 2D Challenges

61. Print matrix in spiral order.
    - **Hint:** Top, right, bottom, left boundaries.
62. Print matrix in zig-zag pattern.
    - **Hint:** Alternate row direction.
63. Print matrix in wave pattern.

- **Hint:** Alternate column direction.

64. Find saddle point.
    - **Hint:** Row minimum and column maximum.
65. Find boundary elements.
    - **Hint:** First/last row or column.
66. Print only border elements.
    - **Hint:** Edge conditions.
67. Find row having maximum even numbers.
    - **Hint:** Count evens per row.
68. Find column having maximum odd numbers.
    - **Hint:** Count odds per column.
69. Replace all negative elements with 0.
    - **Hint:** Traverse and update.
70. Replace all diagonal elements with their squares.
    - **Hint:** Condition `i == j`.

---

## 11 Level 10: Basic 3D Arrays

71. Input and display a 2×2×2 array.

- **Hint:** Use 3 nested loops.

72. Find the sum of all elements in a 3D array.

- **Hint:** Add during traversal.

73. Find the average of all elements.

- **Hint:** Divide by total elements.

74. Find the largest element.

- **Hint:** Track maximum.

75. Find the smallest element.

- **Hint:** Track minimum.

76. Count positive and negative elements.

- **Hint:** Check sign.

77. Count even and odd elements.

- **Hint:** Use `% 2`.

78. Find sum of each layer.
- **Hint:** Fix first index.

79. Find layer with maximum sum.
- **Hint:** Compare layer sums.

80. Search for an element in a 3D array.
- **Hint:** Use 3 nested loops.

---

## 12 Level 11: Advanced 3D Arrays

81. Add two 3D arrays.
- **Hint:** Add corresponding elements.

82. Subtract two 3D arrays.
- **Hint:** Subtract corresponding elements.

83. Multiply every element by a scalar.
- **Hint:** Traverse entire structure.

84. Count zeros in a 3D array.
- **Hint:** Check `== 0`.

85. Find frequency of a given element.
- **Hint:** Compare all positions.

86. Find coordinates of maximum element.
- **Hint:** Store layer, row, column.

87. Find coordinates of minimum element.
- **Hint:** Update indices with min.

88. Reverse elements within each layer.
- **Hint:** Reverse rows/columns.

89. Copy one 3D array into another.
- **Hint:** Triple nested loops.

90. Check whether two 3D arrays are identical.
- **Hint:** Compare all elements.

---

## 13 Challenge Problems

91. Tic-Tac-Toe board using 2D array.
92. Sudoku board representation using 2D array.
93. Seating arrangement system using 2D array.
94. Monthly temperature data (12×30 matrix).
95. Student marks table (Students × Subjects).
96. RGB image representation using 3D array.
97. Voxel (3D cube) representation using 3D array.
98. Warehouse inventory (Floor × Rack × Shelf).
99. Hospital bed management (Building × Floor × Room).
100. Mini spreadsheet using a 2D array.

These 100 problems take a student from **basic matrix traversal** to **advanced 2D/3D array manipulation**, while requiring only arrays, loops, conditions, and operators.

## 14 Advanced Array-Based Problems (Beyond Basic 1D, 2D, and 3D Arrays)

**Prerequisites:** Arrays, loops, conditions, operators, `printf`, `scanf`

These problems emphasize **mathematical thinking, algorithms, patterns, and problem-solving** rather than just array traversal.

---

## 15 Section A: Mathematical Arrays

1. Find the sum of all prime numbers in an array.
    - **Hint:** Check primality for each element.
2. Find the product of all prime numbers in an array.
    - **Hint:** Multiply only primes.
3. Count Armstrong numbers in an array.
    - **Hint:** Apply Armstrong logic element-wise.
4. Count Perfect numbers in an array.
    - **Hint:** Sum proper divisors.
5. Count Palindrome numbers in an array.
    - **Hint:** Reverse each number.
6. Find the GCD of all elements.
    - **Hint:** Repeated Euclidean algorithm.
7. Find the LCM of all elements.
    - **Hint:** Use GCD relation.

8. Find the sum of factorials of all elements.
    - **Hint:** Compute factorial per element.
9. Replace each element by its factorial.
    - **Hint:** Update array values.
10. Replace each element by the sum of its digits.
    - **Hint:** Use `%10` and `/10`.

---

## 16 Section B: Pattern Recognition

11. Check if the array forms an arithmetic progression.
    - **Hint:** Common difference must be constant.
12. Check if the array forms a geometric progression.
    - **Hint:** Common ratio must be constant.
13. Find the missing term in an arithmetic progression.
    - **Hint:** Compare expected difference.
14. Find the missing term in a geometric progression.
    - **Hint:** Compare ratios.
15. Find the next term in a sequence.
    - **Hint:** Observe the pattern.
16. Check if the array is strictly increasing.
    - **Hint:** Every element $>$ previous.
17. Check if the array is strictly decreasing.
    - **Hint:** Every element $<$ previous.
18. Check if the array is bitonic.
    - **Hint:** First increasing then decreasing.
19. Find the longest increasing segment.
    - **Hint:** Maintain current length.
20. Find the longest decreasing segment.
    - **Hint:** Similar logic.

---

## 17 Section C: Frequency and Counting

21. Find the mode (most frequent element).
    - **Hint:** Frequency counting.
22. Find all modes.
    - **Hint:** Multiple elements may share highest frequency.
23. Find the median.
    - **Hint:** Sort first.
24. Find the range.
    - **Hint:** Max − Min.
25. Find frequency distribution table.
    - **Hint:** Count occurrences.
26. Find the element appearing exactly twice.
    - **Hint:** Frequency = 2.
27. Find the element appearing exactly three times.
    - **Hint:** Frequency = 3.
28. Print elements occurring more than n/2 times.
    - **Hint:** Majority element concept.
29. Find the least frequent element.
    - **Hint:** Minimum frequency.
30. Find all unique elements.
    - **Hint:** Frequency = 1.

---

## 18 Section D: Subarray Problems

31. Find the maximum sum subarray.
    - **Hint:** Check all possible subarrays.
32. Find the minimum sum subarray.
    - **Hint:** Similar logic.
33. Count subarrays with sum equal to a given number.
    - **Hint:** Nested loops.
34. Find longest subarray with positive sum.
    - **Hint:** Track lengths.

35. Find longest subarray with even sum.
    - **Hint:** Compute sum of subarrays.
36. Find shortest subarray with sum greater than K.
    - **Hint:** Compare lengths.
37. Print all subarrays.
    - **Hint:** Three loops.
38. Find sum of every subarray.
    - **Hint:** Generate subarrays.
39. Find product of every subarray.
    - **Hint:** Nested loops.
40. Count total number of subarrays.
    - **Hint:** Formula involving n.

---

## 19 Section E: Pair and Triplet Problems

41. Find pairs whose sum equals K.
    - **Hint:** Compare every pair.
42. Find pairs whose difference equals K.
    - **Hint:** Nested loops.
43. Find pairs whose product equals K.
    - **Hint:** Check multiplication.
44. Find triplets whose sum equals K.
    - **Hint:** Three loops.
45. Find triplets whose product equals K.
    - **Hint:** Three loops.
46. Find pair with maximum product.
    - **Hint:** Largest numbers matter.
47. Find pair with minimum product.
    - **Hint:** Consider negatives.
48. Find pair with maximum difference.
    - **Hint:** Max − Min.
49. Count all valid pairs.

- **Hint:** Nested loops.

50. Count all valid triplets.
    - **Hint:** Triple nested loops.

---

## 20 Section F: Matrix-Like Problems Using 1D Arrays

51. Store a polynomial and evaluate it.
    - **Hint:** Coefficients in an array.
52. Add two polynomials.
    - **Hint:** Add corresponding coefficients.
53. Subtract two polynomials.
    - **Hint:** Same idea.
54. Multiply polynomial by scalar.
    - **Hint:** Multiply coefficients.
55. Differentiate a polynomial.
    - **Hint:** Power $\times$ coefficient.
56. Integrate a polynomial.
    - **Hint:** Divide coefficient by power+1.
57. Evaluate polynomial for x.
    - **Hint:** Use powers of x.
58. Find degree of polynomial.
    - **Hint:** Highest non-zero coefficient.
59. Print polynomial in standard form.
    - **Hint:** Traverse coefficients.
60. Find roots by trial values.
    - **Hint:** Substitute values.

---

## 21 Section G: Number-Theory Arrays

61. Generate first N Fibonacci numbers.
    - **Hint:** Store previous values.
62. Generate first N prime numbers.

- **Hint:** Prime checking.

63. Store squares of first N numbers.
- **Hint:** i*i.
64. Store cubes of first N numbers.
- **Hint:** i*i*i.
65. Generate triangular numbers.
- **Hint:** Running sum.
66. Generate Pascal's triangle in arrays.
- **Hint:** Previous row values.
67. Store powers of 2.
- **Hint:** Repeated multiplication.
68. Generate Collatz sequence.
- **Hint:** Even/odd rules.
69. Generate perfect numbers up to N.
- **Hint:** Divisor sums.
70. Generate Armstrong numbers up to N.
- **Hint:** Digit powers.

---

## 22 Section H: Simulation Problems

71. Simulate a queue using an array.
- **Hint:** Front and rear indices.
72. Simulate a stack using an array.
- **Hint:** Top variable.
73. Simulate a voting system.
- **Hint:** Count votes.
74. Simulate a bank transaction ledger.
- **Hint:** Store transactions.
75. Simulate inventory stock.
- **Hint:** Add/remove quantities.
76. Simulate exam scores and rankings.
- **Hint:** Sort marks.

77. Simulate attendance tracking.
- **Hint:** 1 = Present, 0 = Absent.

78. Simulate a cricket scoreboard.
- **Hint:** Store runs.

79. Simulate a temperature logger.
- **Hint:** Daily readings.

80. Simulate a simple calculator history.
- **Hint:** Store results.

---

## 23 Section I: Higher-Dimensional Thinking

81. Sum all elements of a 4D array.
- **Hint:** Four nested loops.

82. Find maximum in a 4D array.
- **Hint:** Track max.

83. Search an element in a 4D array.
- **Hint:** Four loops.

84. Copy one 4D array into another.
- **Hint:** Corresponding positions.

85. Compare two 4D arrays.
- **Hint:** Check every element.

86. Count zeros in a 4D array.
- **Hint:** Traverse all dimensions.

87. Compute layer-wise sums in a 4D array.
- **Hint:** Fix one dimension.

88. Find average of a 4D array.
- **Hint:** Sum / total elements.

89. Reverse elements in a 4D array.
- **Hint:** Traverse backward.

90. Flatten a 4D array into a 1D array.
- **Hint:** Store sequentially.

---

## 24 Challenge Problems

91. Implement Bubble Sort from scratch.
92. Implement Selection Sort from scratch.
93. Implement Insertion Sort from scratch.
94. Implement Binary Search.
95. Merge two sorted arrays.
96. Find intersection of two arrays.
97. Find union of two arrays.
98. Implement Sparse Matrix representation using arrays.
99. Implement Pascal's Triangle.
100. Implement a Mini Spreadsheet using arrays.

These problems are the natural bridge between **basic arrays** and future topics such as **functions, pointers, structures, dynamic memory, data structures, algorithms, and competitive programming**.